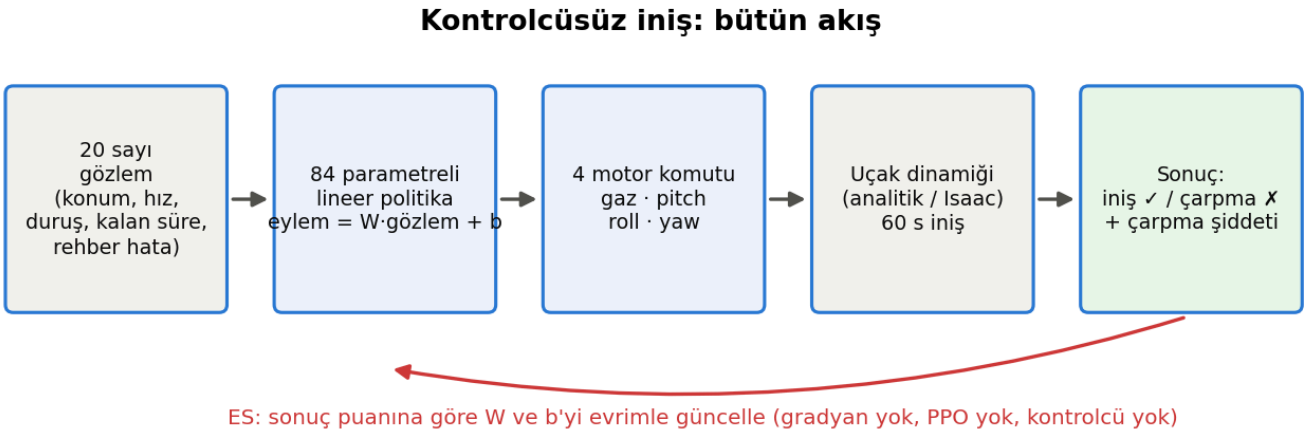
Kontrolcüsüz iniş: evrimsel strateji (ES) nasıl çalışıyor

LunarSim · Apollo LM · Isaac Sim 6.1 · 12 Ekim 2026 · tüm sayılar `out/evo/` dosyalarından okunmuştur

%91.1	%52.6	79–5	84
ES politikası, Isaac orbit_descent 175/192 iniş	ZEM/ZEV kontrolcüsü 101/192 iniş	McNemar (ES-only / kontrolcü-only) p = 3.4e-18	parametre, kontrolcü yok, gradyan yok

1. Ne yapıldı

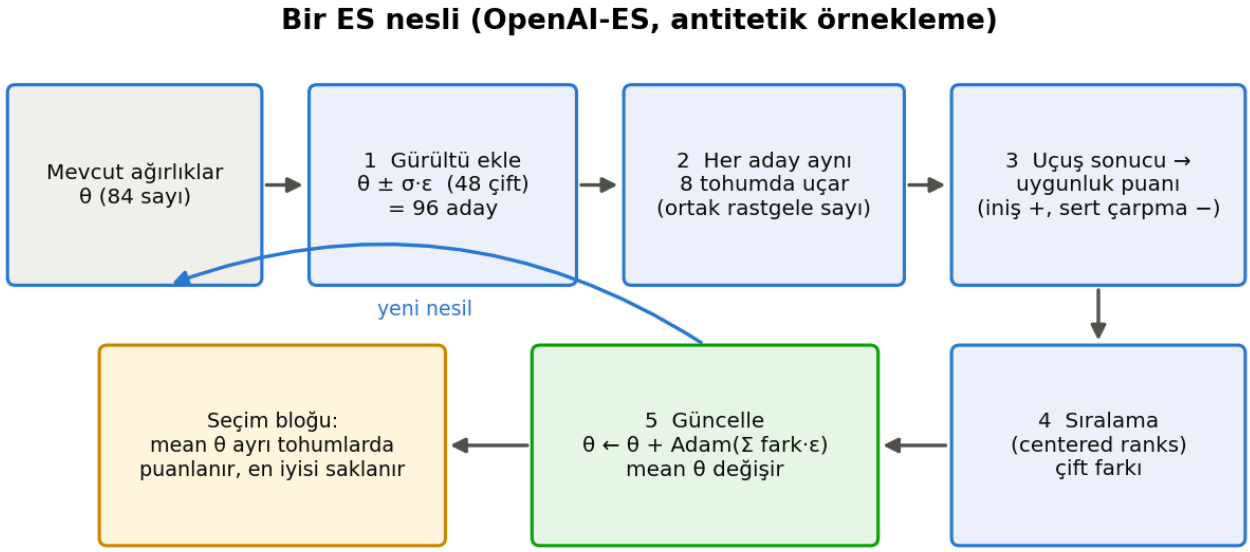
Amaç: iniş için hiçbir kontrol yasası, demonstrasyon ya da taklit öğrenmesi kullanmadan, yalnızca uçuşun **sonucundan** öğrenen bir algoritma bulmak. Yöntem: 84 sayılık lineer bir politikayı, PPO yerine **evrimsel strateji** ile eğitmek. Politika doğrudan gaz ve üç eksen RCS torkunu komut ediyor; ZEM/ZEV kontrolcüsü hiçbir yerde uçmuyor.



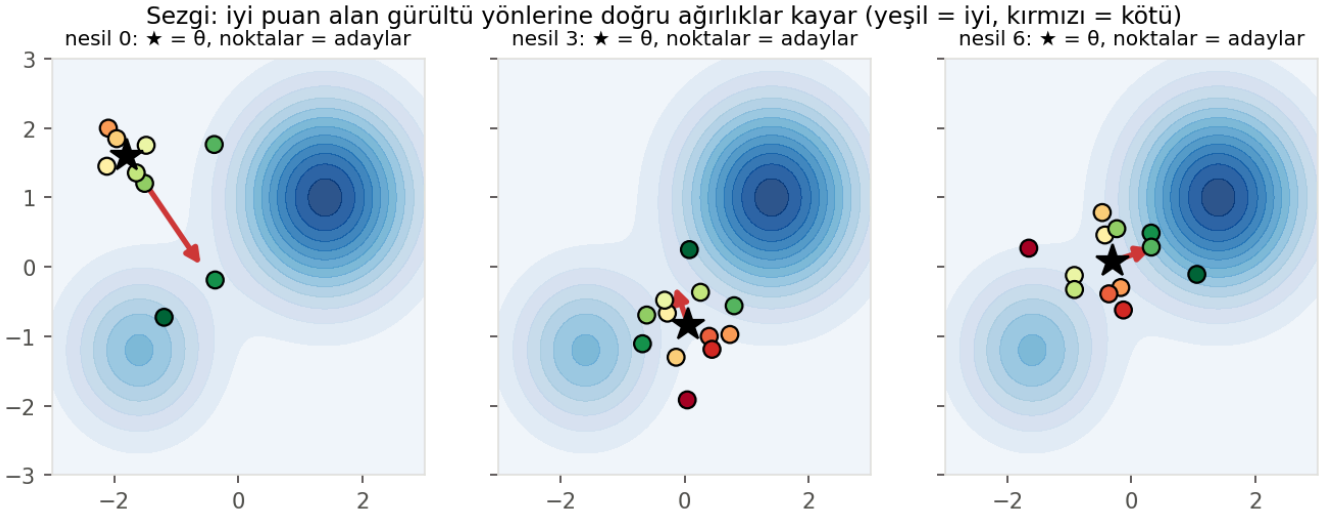
Şekil 1. Bütün akış: gözlem → lineer politika → komutlar → uçuş → sonuç; ES sadece sonuca bakıp ağırlıkları günceller.

2. ES nasıl çalışıyor

PPO'nun aksine ES bir bölümü **tek bir sayıyla** puanlar ve zaman içinde kredi dağıtmaz. Bu, 800 adımlık uzun ufukta terminal ödülün sönümlenmesi sorununu ortadan kaldırır. Her nesilde ağırlıklara bir kez gürültü eklenir; keşif bölüm boyunca tutarlıdır.



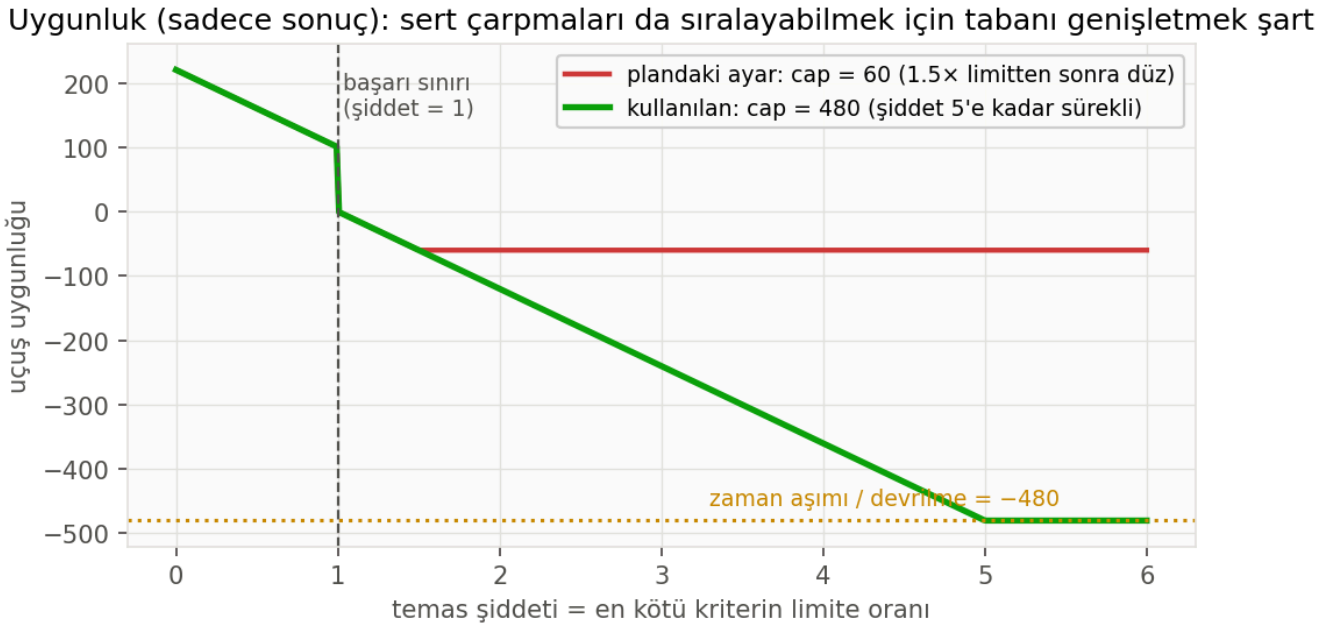
Şekil 2. Bir nesil: 48 antitetik çift (θ±σε), ortak tohumlarda uçuş, sıralama, Adam ile güncelleme. Nihai aday yalnızca ayrı seçim bloğundan seçilir.



Şekil 3. İki boyutlu oyuncak örnek: iyi sonuç veren gürültü yönleri ağırlıkların hareket yönünü belirler. Gerçekte 84 boyut var, mantık aynı.

3. Uygunluk: sadece sonuç

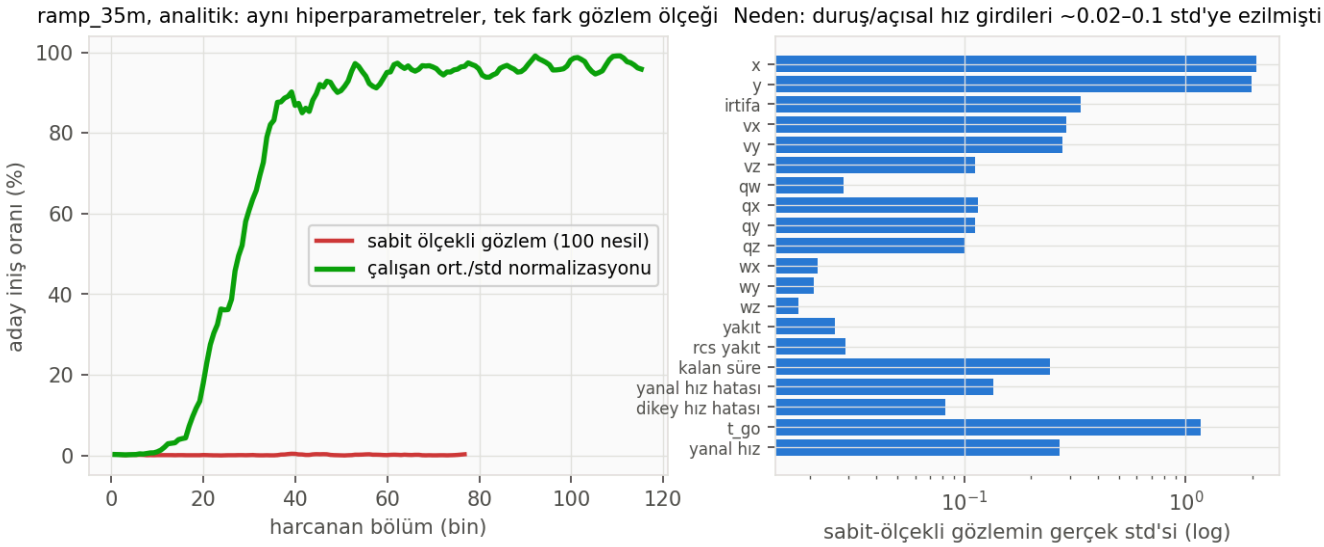
Yoğun şekillendirme ödülü bu projede iniş oranıyla *ters* sıraladığı için kullanılmadı; yalnızca temas sonucu puanlanıyor. Mevcut `RewardWeights` alanlarıyla kuruldu, yeni ödül kodu yazılmadı.



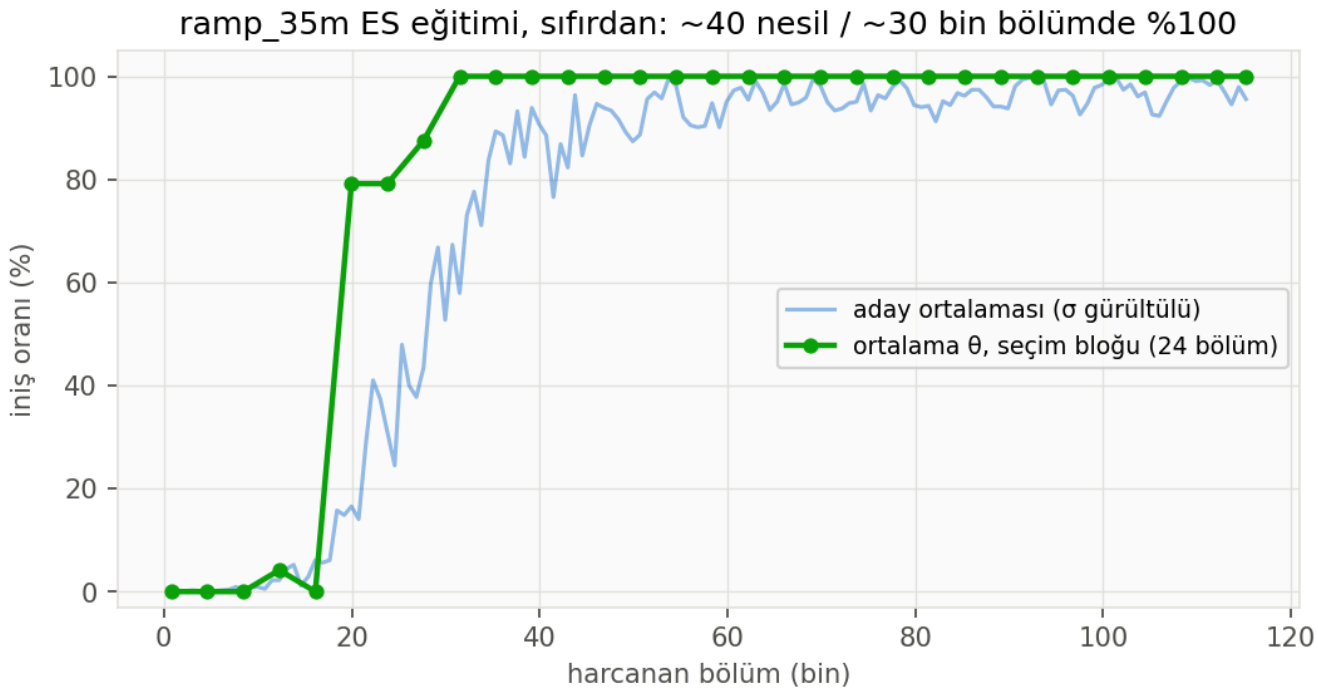
Şekil 4. Planın önerdiği `touchdown_penalty_cap=60` her sert çarpmayı (şiddet ≥ 1.5) aynı −60'a eşitliyor; ES'in sinyali sıfır. Kullanılan ayar 480. Zaman aşımı da −480 olduğu için hover (asılı kalma) çarpmadan daha iyi puan almıyor.

4. İşi çözen tek şey: gözlem normalizasyonu

İlk denemelerde ES hiç öğrenmedi (üç farklı σ/lr ayarı denendi, aday iniş oranı en iyi %0.9'da kaldı). Aynı hiperparametrelerle yapılan temiz ablasyonda (yalnızca `--obs-norm 0`) normalizasyonsuz koşu 100 nesilde %0.4'ün altında kaldı, seçim bloğu %0; normalizasyonlu koşu ~45. nesilde %89, 100. nesilde %98 aday oranına ulaştı. Sebep: gözlem sabit ölçeklerle bölünüyordu (yanal hız /30, açısal hız /2…); anlamlı geri besleme için ağırlıkların 10–100 mertebesine çıkması gerekiyordu, σ=0.1 ile bu binlerce nesil demek. Çalışan ortalama/std ile normalizasyon eklenince ~40 nesilde %0'dan %100'e çıktı.



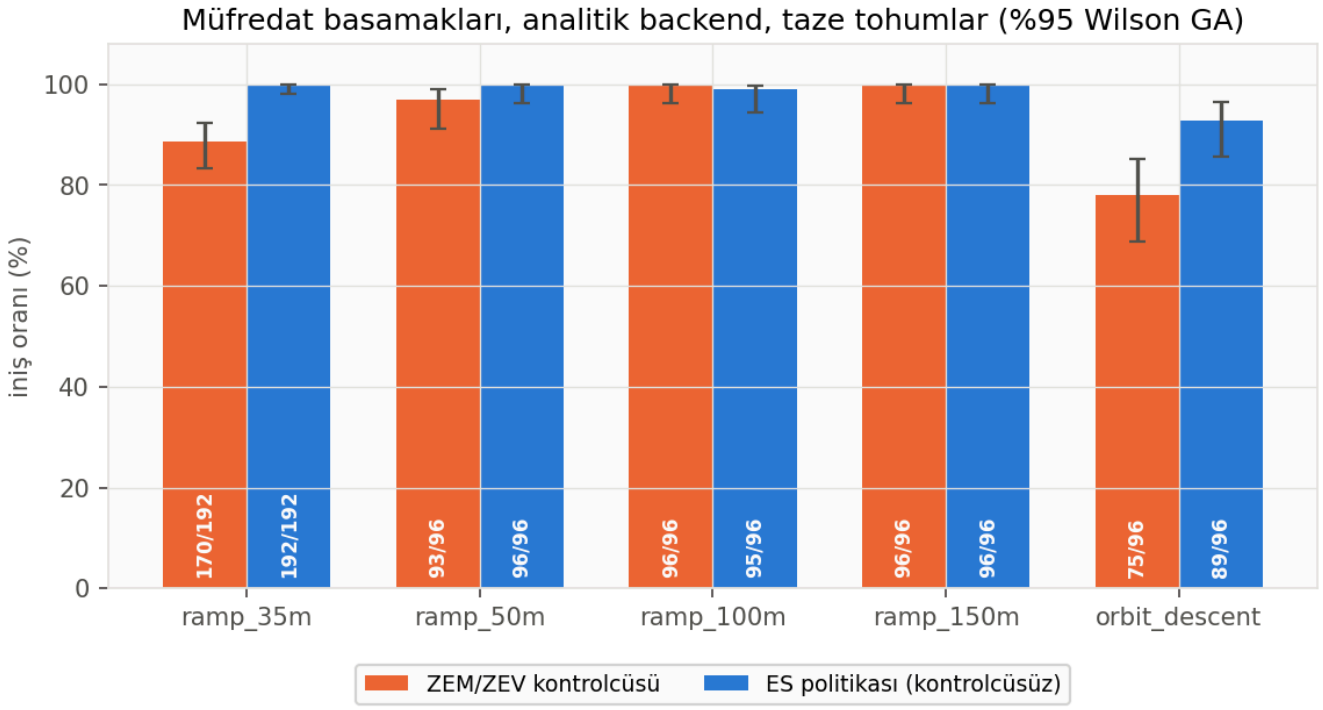
Şekil 5. Solda aynı ES ve aynı hiperparametreler (σ=0.1, lr=0.05, 48 çift, 8 bölüm/aday), yalnızca gözlem ölçeği farklı. Sağda neden: sabit ölçekten sonra quaternion ve açısal hız girdilerinin gerçek std'si ~0.02–0.1, hızlar ~0.1–0.3 (birim varyanstan çok küçük).



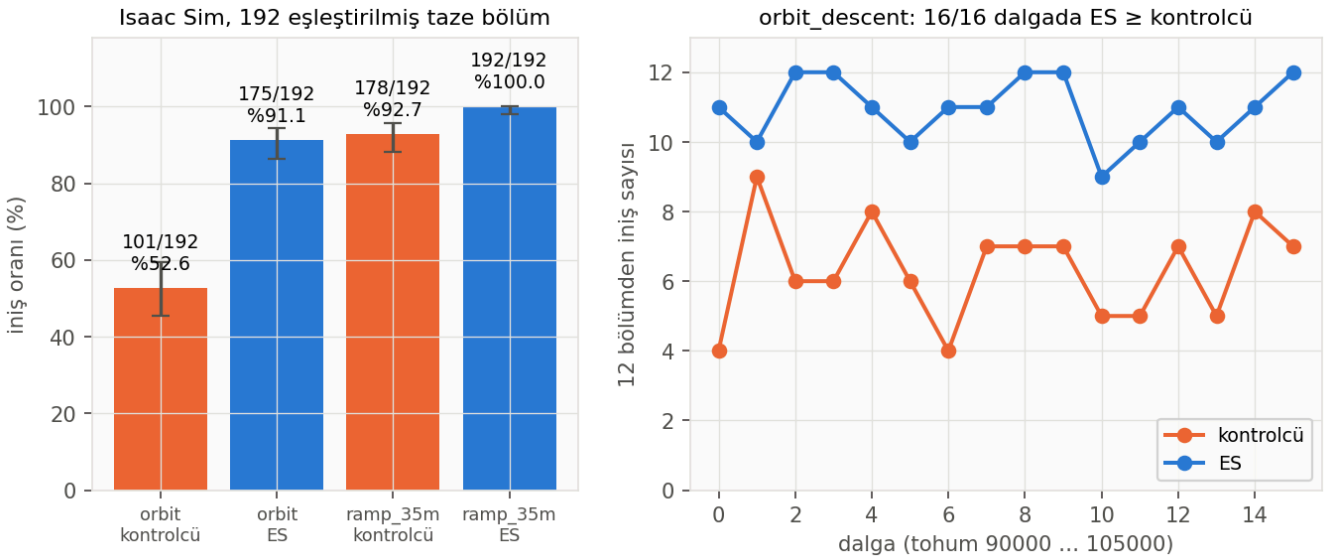
Şekil 6. ramp_35m'de sıfırdan eğitim eğrisi. Aday ortalaması gürültülü adayları içerdiği için ortalama θ'dan düşüktür.

5. Müfredat ve sonuçlar

`ramp_35m → ramp_50m → ramp_100m → ramp_150m → orbit_descent`; her basamak bir öncekinin ağırlıklarından başlar (warm-start), seçim bloğu ≥ %97 olunca durur. orbit_descent warm-start ile 0. nesilde bile %53 aday inişi verdi.



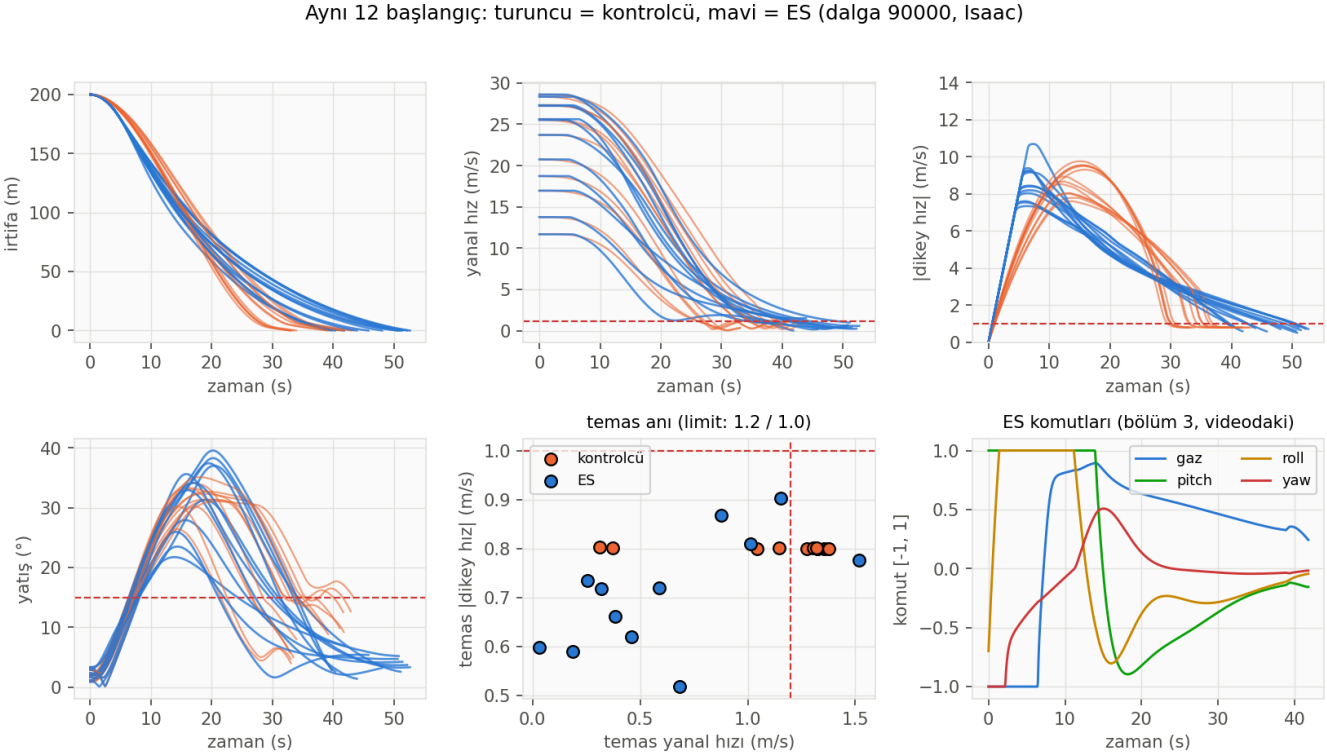
Şekil 7. Analitik backend'de her basamak: ES her basamakta kontrolcüyle eşit ya da daha iyi; orbit_descent'te belirgin üstün.



Şekil 8. Isaac Sim, eşleştirilmiş taze tohumlar. orbit_descent tohum bloğu (90000–105000) eğitim ve seçimde kullanılmadı.

Isaac orbit_descent, 192 bölüm	**iniş**	**%95 GA (Wilson)**	**sonlanma**
ZEM/ZEV kontrolcüsü	101/192 (%52.6)	[45.6, 59.5]	91 çarpma
ES (kontrolcüsüz)	**175/192 (%91.1)**	[86.3, 94.4]	17 çarpma, 0 devrilme, 0 zaman aşımı

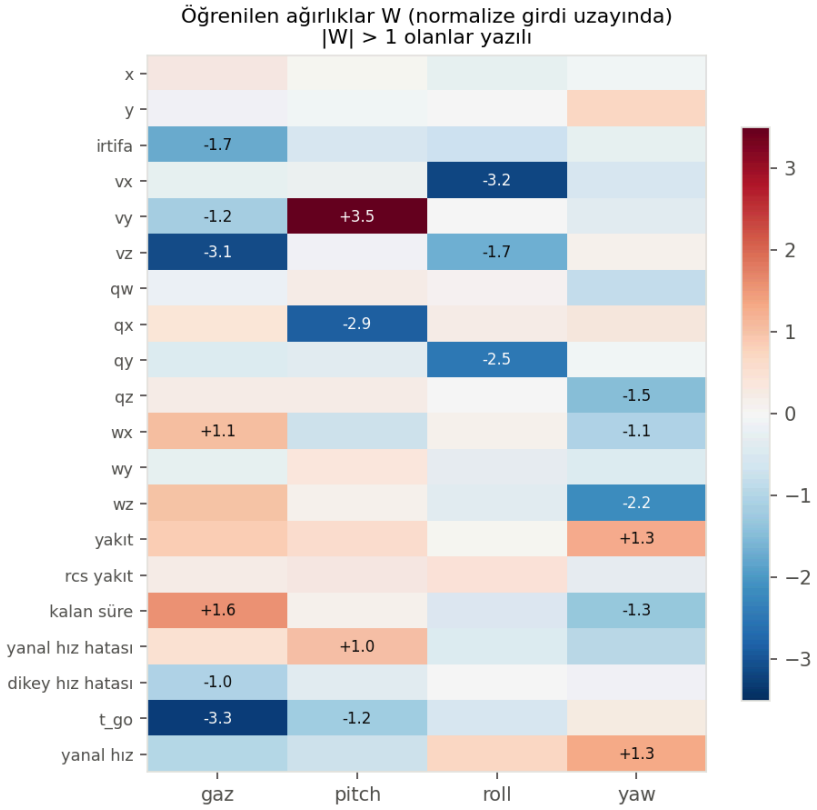
6. Politika uçarken ne yapıyor



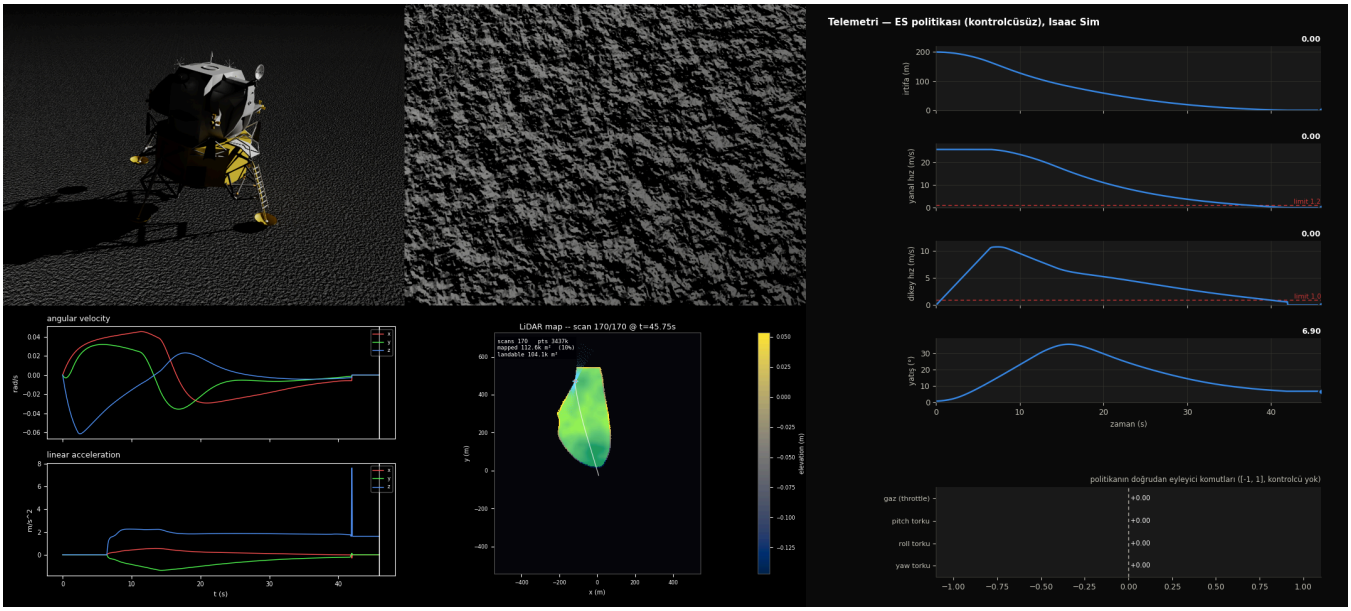
Şekil 9. Aynı 12 başlangıçta kontrolcü (turuncu) ve ES (mavi). Sağ altta videodaki bölümün komutları.

Ağırlıklardan okunan davranış (Şekil 10): **gaz** kalan uçuş süresi (t_go) ve dikey hızdan negatif geri besleme alır, yani hızlı iniyorsa gazı açar. **Pitch/roll** yanal hızla eğimin zıt işaretli birleşimine

bağlıdır: bir yönde hız varsa o yöne ters eğilip yanal hızı söndürür, eğim büyüyünce ters tork verir. Bu, kademeli bir PD geri beslemedir. **Yaw** esas olarak yaw hızını sönümler.



Şekil 10. Öğrenilen W matrisi. Dört çıktı için en büyük girdiler t_go, vz, vx/vy, quaternion bileşenleri ve açısal hızlar.



Şekil 11. Videonun son karesi (`es_orbit_lidar_dashboard.mp4`): şase ve nav kamerası, IMU, hesaplanmış LiDAR haritası ve ES komutları. Kontrolcü bu bölümde çarpıyor, ES iniyor. Kaydın son yaklaşık 4 saniyesi temas sonrası bekleme olarak eklenmiştir (araç ortamın temas modelinde zaten temasta durur); ivme grafiğindeki tek sivri çıkıntı temas anında hızın sıfırlanmasının sonlu farkıdır.

7. Dürüst sınırlar

- **Eğitim analitik backend'de yapıldı.** Isaac'te yalnızca sıfır-atış (zero-shot) aktarım doğrulandı; Isaac'te ES eğitimi yapılmadı.
- **Gözlem "ham" değil.** 16–19. slotlar kılavuz alanına göre hız hatası, t_go ve yanal hızı içerir; ödül ağırlıklarından hesaplanır. PPO da aynısını görüyordu; politika t_go'ya dayanıyor. "Kontrolcüsüz" doğru, ama girdi tasarımında rehber bilgi var.
- **Eğitim kısa kesildi:** orbit_descent'te 15 nesilde durdu (24 bölümlük seçim bloğu iki kez ≥ %97). Daha uzun eğitim bu oranı yükseltebilir; denenmedi.
- **PPO ile karşılaştırma:** guided-gain PPO %89.6 idi; ES %91.1. 192 bölümle bu fark ayırt edilemez (çözünürlük ~15 puan).
- **17 çarpmanın sebebi** (hangi kriter) henüz ayrıştırılmadı; `ramp_20m` Isaac'te ölçülmedi.
- **Tek ES tohumu.** Plan en az iki tohum istiyordu; tekrarlanabilirlik ölçülmedi.

8. Yeniden üretim

adım	komut
ramp_35m, sıfırdan	`python scripts/evo_analytic_train.py --stage ramp_35m --gens 150 --hidden= --init-scale=0.05 --sigma=0.1 --lr=0.05 --pairs=48 --eps-per-cand=8 --out out/evo/an_n1`
müfredat	`out/evo/ladder.sh` (ramp_50m → orbit_descent, `--init-npz` ile warm-start, `--stop-land=0.97`)
Isaac doğrulama	`python scripts/evo_isaac_eval.py an_orbit_descent_best.npz --stage orbit_descent --waves 16 --wave-seed0 90000`
video	`scripts/evo_record_isaac.py` → `scripts/isaaclab_replay_recording.py` → `scripts/evo_compose_dashboard_video.py`